

Assignment: Real-World ETL from S3 to DynamoDB

Course: Data Engineering - Cloud Databases & Serverless ETL

Goal: Design and demonstrate a small ETL pipeline where third-party real-world data is uploaded to S3, processed by Lambda, and loaded into DynamoDB.

Submission Format: One PDF report plus screenshots/code files as requested below.

1. Business Scenario

You are a Data Engineer building a serverless ingestion pipeline for an analytics application. Your company receives real-world data from a public third-party API or open dataset. The raw data must first land in Amazon S3. Then an AWS Lambda function should read the data, clean and transform it, and load the final records into Amazon DynamoDB.

Your job is to create a realistic data scenario, design the DynamoDB table, implement the ETL logic, and explain how your system works.

2. Allowed Dataset Options

Choose **one** real-world third-party data source. You may use any public API or public dataset that does not require payment or private credentials.

Option	Example Dataset	Possible Scenario
A	USGS Earthquake API	Earthquake risk monitoring pipeline
B	OpenWeather sample/API data	City weather snapshot ingestion pipeline
C	World Bank API	Country indicator tracking pipeline
D	Public COVID/health CSV dataset	Health analytics ingestion pipeline
E	Open food/product dataset	Product catalog quality pipeline
F	Any approved public CSV/API	Create your own realistic data scenario

3. Required Architecture

```
Third-party data source
-> Google Colab or local Python script
-> Raw file uploaded to S3
-> Lambda processes the S3 file
-> Clean records loaded into DynamoDB
-> Student verifies records in DynamoDB
```

You may trigger Lambda manually from the AWS Console, from Colab, or through an S3 event trigger. For this assignment, manual Lambda invocation is acceptable.

4. Minimum AWS Resources Required

Create the following resources in AWS. Use simple names and include your name or initials to avoid conflicts.

Resource	Requirement
S3 Bucket	Store raw data file under a prefix such as raw/
Lambda Function	Read the S3 object, process records, and write items to DynamoDB
DynamoDB Table	Store clean transformed records
IAM Role	Lambda role with least-privilege permissions for S3 read and DynamoDB write

5. DynamoDB Table Design

Design a DynamoDB table based on your selected data. At minimum, your table must have a meaningful partition key.

Example: If using earthquake data, the table can be named earthquake_events with partition key event_id.

Your table should store at least **6 useful attributes** per record, such as id, timestamp, category, location, numeric measurement, risk/status label, and processed timestamp.

6. ETL Requirements

Extract

Fetch or download a real-world dataset using Python. Save the raw data as CSV or JSON. Upload this raw file to S3 under a prefix such as raw/.

Transform

Your Lambda function must apply at least **four** transformations or validations. Examples:

- remove records with missing ID or timestamp
- convert numeric values to proper types
- standardize text fields
- create a risk/status/category column
- convert timestamp format
- keep only records matching a condition
- add processed_at_utc

Load

Insert the clean records into DynamoDB. Use the partition key to avoid duplicate records when the same file is processed again.

7. Suggested Example: Earthquake Risk ETL

This is only an example. You may choose another dataset.

```
Dataset: USGS Earthquake API
Scenario: Build an earthquake risk monitoring pipeline
Raw file in S3: raw/earthquakes.json
DynamoDB table: earthquake_events
Partition key: event_id
```

Sample transformed attributes:

- event_id
- magnitude
- place
- event_time
- longitude
- latitude
- depth_km
- risk_level
- processed_at_utc

Possible risk rule: High if magnitude ≥ 6 , Medium if magnitude ≥ 4.5 , otherwise Low.

8. Lambda Output Requirement

Your Lambda function should return an audit summary similar to this:

```
{
  "source_file": "raw/earthquakes.json",
  "target_table": "earthquake_events",
  "input_records": 120,
  "clean_records": 108,
  "rejected_records": 12,
  "inserted_records": 108,
  "processed_at_utc": "2026-06-17T10:00:00Z"
}
```

9. Security Requirements

Your implementation must follow these rules:

- Do not make your S3 bucket public.
- Do not hardcode AWS access keys in your notebook or Lambda code.
- Use IAM permissions with least privilege.
- Do not commit credentials, .env files, or raw datasets to GitHub.
- Delete unused AWS resources after submission if instructed.

10. Submission Checklist

Deliverable	Required Evidence
Scenario write-up	Explain the business/data problem in 5-8 lines
Dataset source	Mention the public dataset/API used and why it is useful
Architecture diagram	Show source -> S3 -> Lambda -> DynamoDB
S3 screenshot	Show raw file uploaded to S3
Lambda screenshot/code	Show Lambda function or important code section
DynamoDB screenshot	Show inserted clean records/items
Audit output	Show Lambda response summary
Transformation explanation	List at least four cleaning/transformation rules
Reflection answers	Answer the questions in Section 11

11. Reflection Questions

Answer these briefly in your submission:

1. Why is S3 a good landing zone for raw files?
2. Why is Lambda suitable for this small ETL workload?
3. Why did you choose your DynamoDB partition key?
4. What is the difference between storing this data in DynamoDB vs RDS?
5. Why should we avoid scanning large DynamoDB tables in production?
6. What IAM permissions did your Lambda function need?

12. Evaluation Rubric

Criteria	Marks
Clear real-world scenario and dataset choice	10
Correct S3 raw data upload	10
Working Lambda ETL logic	25
Meaningful transformations and validations	20
Correct DynamoDB table design and loaded records	15
Security and least-privilege thinking	10
Screenshots, explanation, and reflection answers	10
Total	100

13. Bonus Challenge

Add one of the following bonus features:

- Store rejected records in a separate DynamoDB table or S3 file.
- Add a Global Secondary Index for a common query pattern, such as customer_id, city, country, or risk_level.
- Add a processed file tracking table to prevent duplicate file processing.
- Create a small query script in Colab to read back items from DynamoDB.

Reminder: Keep the project small. The goal is to demonstrate the complete ETL flow, not to process millions of records.